

Physically-Based Volumetric Rendering of Astronomical Nebulae

Giorgia Savo giorgias@kth.se
Personal Number: 20040115-T485

Francesco Filippo Stefanutti ffst@kth.se
Personal Number: 200203278858

DH2323 — Computer Graphics
KTH Royal Institute of Technology, School of EECS
May 13, 2026



1. Introduction and Motivation

Nebulae represent some of the most interesting and visually rich phenomena in the known universe. Producing a convincing nebula image requires simultaneously solving problems that are in the heart of modern computer graphics: **physically-grounded light transport in participating media, procedural generation of density and integration along rays (raymarching)**.

This project aims to develop a CPU-based renderers in C++, each targeting a different scientific paradigm for nebula generation.

2. Scientific Background

Unlike many engineering domains, **there is no single canonical pipeline for the rendering of astrophysical nebulae**. The field has evolved through a collection of independent contributions spanning physically-based rendering, procedural content generation, and scientific visualisation, each addressing different aspects of the problem.

The following pipeline was derived by surveying several related works. Their methods have been assembled into a unified workflow that reflects current professional practice:

I. **Step1 - Construction of the Volumetric Density Grid:** the first step is to obtain a **three-dimensional density field** that represents the macroscopic shape of the nebula. Two main distinct approaches exist, differing fundamentally in their data source:

- **Option A - Procedural Mesh-Based:** in this approach, the macroscopic shape of the nebula is first modelled as a triangulated mesh, either artist-controlled or derived from observational data. The mesh is then converted into grid `float density[N][N][N]` via ray casting, a technique also employed in production VFX by Sony Pictures Imageworks and DreamWorks Animation [3].

One of the most established tools following this paradigm is the SHAPE software [6], which implements a **Forward Modelling** approach: the astronomer hypothesises a 3D morphology, renders it into a synthetic 2D image and FITS file simulating a terrestrial telescope observation, and iteratively compares the result against real telescope data until convergence. The converged model is then exported as a triangulated mesh or FITS datacube, which is subsequently voxelised for use in the following steps.

- **Option B - Observational Data-Driven:** A fundamentally different approach reconstructs the density field directly from **real spectroscopic telescope data**, with no intermediate geometric model. Instruments such as SITELLE or JWST measure, for each sky pixel, a full spectrum from which a radial velocity is derived via the **Doppler effect**, as explained in [5]. This velocity is then converted into a spatial coordinate along the line of sight, producing a three-dimensional density grid that directly encodes the measured structure of the object, including asymmetries, filaments and turbulent voids. The data is stored as a **FITS** (Flexible Image Transport System) **datacube**: a three-dimensional array where two axes correspond to sky coordinates (right ascension and declination) and the third encodes radial velocity, interpreted as depth.

In both cases, the output of step 1 is a **density grid**, common input to all subsequent steps.

The current industry standard for representing volumetric data is **OpenVDB** [7], a sparse hierarchical data structure that allocates memory only where density is non-zero. This is essential for large-scale simulation data where the majority of the volume is empty.

- II. **Step 2 - fBm/Perlin Noise for filaments and turbulence:** already in [1] there are references to methods for introducing natural/fractal variation of signals: he cites the work of Richard F. Voss, who in the early 1980s formalized the generation of $\frac{1}{f}$ noise (pink noise), producing structures closely resembling natural phenomena. The decisive contribution to computer graphics came with **Perlin Noise** [2]. He introduced a continuous noise function, from which several methods have been developed to construct textures and fractal patterns (such as fBm).

Sony Pictures Imageworks and **Dreamworks Animation** [3] have employed exactly this technique: the shape of volumetric objects is defined by the mesh geometry, while fractal patterns (fBm built on Perlin noise) add a cloud-like appearance. The fBm is constructed as a **sum of layered procedural noise** functions, with frequency and amplitude varying from layer to layer.

This step applies identically regardless of whether the density grid came from Option A or Option B.

- III. **Step 3 - Volume Rendering and RTE:** the formal justification of ray marching through the Radiative Transfer Equation originates from [1]. This work introduces algorithms for rendering objects represented as density fields within a voxel grid (e.g., clouds, fog, flames, dust, and particle systems), deriving the equations of light scattering and proposing a solution to three-dimensional radiative scattering in computer graphics. It represents a foundational reference for this approach. The discrete formulation of this method, including early termination based on transmittance, is clearly explained and implemented in Scratchapixel .
- IV. **Step 4 - Emission color model:** Until now we have described *how* we march (RTE) but not *how we assign the color* to the sampled gas. A physically motivated approach to colour assignment within the ray marching loop is proposed by Carlson [4]. Rather than assigning a fixed colour to the accumulated density, the method derives the emission intensity of each chemical element from the **local ionization state** of the gas, which in turn depends on the cumulative density between the sample point and the ionizing star.

3. Our Project

Our first idea for the project is to **implement an offline volumetric renderer**. Since we are not astronomers and we are not interested in the techniques for inferring the shape of nebulae, our approach is more focused on the rendering part rather than the mesh reconstruction one.

The rendering process is divided into six main stages:

1. **FITS Input:** FITS files are downloaded from NASA/ESO datasets using **astroquery**, a Python library.
2. **Voxelization:** the FITS files are processed to extract different volumetric grids, which are then passed to the C++ raymarcher for the subsequent stages.
3. **Fractal Noise:** Fractional Brownian Motion (fBm), built on top of Perlin Noise, is injected at sampling time to add filaments and fine-scale turbulence, without additional memory cost.
4. **Ray Marching:** for each output pixel, a ray is cast through the volume. Density is sampled at regular intervals and accumulated using the discrete radiative transfer equation.
5. **Color Application:** some of the volumetric grids extracted in Step 2 are used to determine the emission colour of each sample. For nebulae, colour is physically driven by the presence of specific chemical species, such as hydrogen or oxygen, within each voxel.
6. **Output and Navigation:** the rendered buffer is saved in PPM format. The user is additionally given the ability to navigate the volume freely, observing the nebula from arbitrary viewpoints, including from within.

It is worth noting that the primary goal of this project is not to produce a photorealistic replica of a nebula, for that, telescopes already exist. Rather, the aim is to **develop an interactive renderer that allows the user to move freely around and through the nebular volume**, exploring its three-dimensional structure from perspectives that no telescope can offer.

4. Aimed grade and deadline

We aim to achieve the highest grade.

Note that, since Giorgia is an Erasmus student, receiving the final grade before the start of July 2026 would be greatly appreciated.

References

- [1] J. T. Kajiya and B. P. Von Herzen, Ray Tracing Volume Densities, *Proceedings of SIGGRAPH 1984 (ACM Computer Graphics)*, Vol. 18, No. 3, pp. 165–174, 1984. DOI: 10.1145/964965.808594
- [2] K. Perlin, “An Image Synthesizer,” *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985.
- [3] M. Wrenninge and N. B. Zafar, *Volumetric Methods in Visual Effects*, SIGGRAPH 2010 Course Notes, 2010. Available at: [link](#)
- [4] A. Carlson, A rendering method for simulated emission nebulae, Bachelor’s thesis, University of Gävle, 2011. Available at: [link](#)
- [5] M. Charlebois, L. Drissen, A.-P. Bernier, F. Grandmont, and L. Binette, “A hyperspectral view of the Crab Nebula” *The Astronomical Journal*, vol. 139, no. 5, p. 2083, 2010. doi:10.1088/0004-6256/139/5/2083
- [6] W. Steffen, N. Koning, S. Wenger, C. Morisset, and M. Magnor, “Shape: A 3D Modeling Tool for Astrophysics,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 4, pp. 454–465, 2011. doi:10.1109/TVCG.2010.62 SHAPE software available at: [shape](#)
- [7] K. Museth, “VDB: High-Resolution Sparse Volumes with Dynamic Topology,” *ACM Transactions on Graphics*, vol. 32, no. 3, art. 27, pp. 27:1–27:22, 2013. doi:10.1145/2487228.2487235